

## FIT1047 Introduction to computer systems, networks and security - S2 2025

### Assignment 2 – MARIE Programming

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>    | This assignment is mainly about programming in the MARIE assembly language. This will allow students to demonstrate their comprehension of the fundamental ways a processor works. In addition, basic knowledge about processes, memory and I/O is assessed in an in-class interview. The assignment relates to Unit Learning Outcomes 2, 3 and 4.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Your tasks</b> | <b>Part 1: Submit your reflections (Week 4 - 6).</b> See details below.<br><b>Part 2:</b> Disassemble and add comments to a MARIE program.<br><b>Part 3:</b> Write a MARIE program that can display bitmap alphabets.<br><b>Part 4:</b> In-class in-person interview (Week 8 applied session).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Value</b>      | 25% of your total marks for the unit.<br>The assignment is marked out of 60 marks.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Word Limit</b> | See individual instructions                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Due Date</b>   | <b>Part 1–3: 11:55 PM Friday 5 September 2025</b><br><b>Part 4: Interview conducted during your official allocated Applied Session in Week 8</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <b>Submission</b> | <ul style="list-style-type: none"> <li>• Via Moodle Assignment Submission.</li> <li>• <b>DRAFT</b> upload confirmation email from Turnitin is not a submission. You must click the submit button to accept terms and conditions in Moodle. Note that <b><u>DRAFT submissions will not be assessed.</u></b></li> <li>• <b>Once the submission is confirmed, any requests to revert it back to DRAFT for resubmission will not be accepted.</b> Also, <b>any incorrect, corrupted, empty or wrong file type</b> submission will not be assessed. Please check carefully before confirming your submission.</li> <li>• Turnitin and MOSS will be used for similarity checking of all submissions. <b>Ignore Turnitin warnings or error messages for .mas files or other non-document files.</b></li> <li>• This is an individual assignment (group work is not permitted).</li> <li>• In this assessment, you must <b>not</b> use generative artificial intelligence (AI) to generate any materials or content in relation to the assessment task.</li> <li>• You will need to <b>explain and extend your code</b> in an interview. (Part 4)</li> </ul> |

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Assessment Criteria</b> | <p>Part 1 is assessed based on relevance of the submission to the unit.<br/>Part 2 is assessed based on correctness of the code and the labels/comments.<br/>Part 3 is assessed based on correctness of the code, as well as the documentation/comments.<br/>Part 4 is assessed based on the understanding of the code you have written, and basic knowledge about processes, memory and I/O. See instructions for details.</p> <p>The marking rubric in Moodle shows an individual marks breakdown.</p> <p>If no or insufficient reflections are submitted (Part 1), the overall marks for this assignment are capped at 30 (i.e., 50% of the overall mark).</p> |
| <b>Late Penalties</b>      | <ul style="list-style-type: none"> <li>• 5% deduction per calendar day or part thereof for up to one week</li> <li>• Submissions more than 7 calendar days after the due date will receive a mark of zero (0) and no assessment feedback will be provided.</li> <li>• Late penalty is automatically calculated based on Moodle submission.</li> </ul>                                                                                                                                                                                                                                                                                                             |
| <b>Support Resources</b>   | See Moodle Assessment page                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| <b>Feedback</b>            | <p>Feedback will be provided on student work via:</p> <ul style="list-style-type: none"> <li>• general cohort performance</li> <li>• specific student feedback ten working days post submission</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

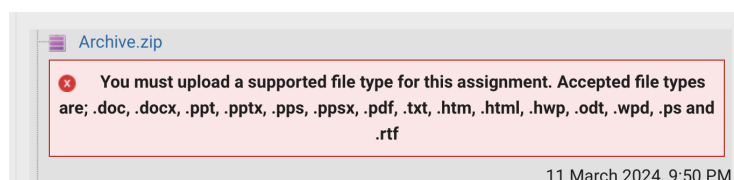
## INSTRUCTIONS

**This assignment has four parts. Make sure you read the instructions carefully.**

**For Part 1**, collect your **reflections** for Weeks 4 - 6 from each week's Ed Lesson and create a single PDF document. **Submit your PDF through the Moodle Assignment 2 Part 1 activity.**

**For Parts 2 and 3**, you need to submit files through the Moodle **Assignment 2 Part 2 and Part 3** activities. Each part requires the submission of **one or two .mas files inside a .zip file.**

**You may see the following error message from Turnitin. You can ignore this error message!**



**Part 4 is an in-class interview** during your allocated Applied Class in Week 8. Instructions will be available in Moodle and communicated via an announcement post.

## Part 1: Reflection (Not marked, but cap on overall mark applies)

Complete your reflection activities for Week 4 to Week 6 in the corresponding Ed Lesson and copy/paste them into a PDF file. Write at least 100 words for each week (relevant and meaningful to the specific week).

**Failure to submit all relevant reflections (missing all submissions or incomplete submissions) will result in your Assignment 2 having a maximum mark of 50%.** For example, if the overall combined mark is 31/60, it will be scaled to 30/60. If the overall combined mark is 28/60 then it will remain as 28/60.

You may use this template:

[https://docs.google.com/document/d/18UIEJQeyarYW1pl8oDEaf--ubCdJ5LDf-9\\_jSLbGxrE/edit?usp=sharing](https://docs.google.com/document/d/18UIEJQeyarYW1pl8oDEaf--ubCdJ5LDf-9_jSLbGxrE/edit?usp=sharing) to write down your reflections.

## Part 2: MARIE Disassembly (15 marks total)

Follow the link on Moodle to access your personalised MARIE memory screenshot for this task.

**Important:** Your memory screenshot is different from the one other students are working on. Only download the file while you are correctly logged into Moodle with your own student account.

### Task 2.1: Disassemble the memory (7 marks)

Based on the memory contents, recreate the MARIE program that corresponds to your personalised memory screenshot. This is called “disassembling” the machine code, since it is the opposite operation of “assembling” the MARIE code into the binary memory contents.

For each memory cell, decode the instruction and (if applicable) the address that the memory cell is encoding. You can make the following assumptions:

- There is exactly one Halt instruction in the code
- Every memory location after the Halt instruction contains data
- Any memory location that contains the value 0 is data (even before the Halt instruction)

Here is an example of a memory screenshot and the corresponding decoded MARIE program:

|     | +0   | +1   | +2   | +3   | +4   | +5   | +6   | +7   | +8   | +9   | +A   | +B   | +C   | +D   | +E   | +F   |
|-----|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|------|
| 000 | 5000 | 3005 | 6000 | 9000 | 7000 | 000A | 0000 | 0000 | 0000 | 0000 | 0000 | 0000 | 0000 | 0000 | 0000 | 0000 |

Disassembled program:

```

Input
Add 005
Output
Jump 000
Halt
DEC 10

```

**Note:** You need to decode the actual instructions. E.g., for the first memory location, HEX 5000 would not be a valid answer. The contents of all memory that follows the Halt instruction is considered to be data. Therefore, DEC 10 is the correct decoding of location 5 (instead of JnS 00A), and HEX 00A would also be correct. You don't need to list all the locations containing zeros starting from address 006 (these will be filled with zeros by the assembler anyway).

**Hints:** You can verify that your disassembled code is correct by entering it into the MARIE simulator, assembling it and comparing the memory contents to the screenshot you started from.

### Task 2.2: Add labels (4 marks)

Now update the program you decoded in Task 2.1. Remove all hard-coded memory addresses by adding labels to replace all memory locations that are used as addresses in the program instructions. Labels should have meaningful names in the context of what the program does (i.e., not just A, B, C). For the example above, this could result in the following program:

```

MainLoop, Input
Add Ten
Output
Jump MainLoop
Halt
Ten, DEC 10

```

**Hints:** All data used by the code can be interpreted as integer numbers, so it's probably best to use the *Decimal* output mode. Run the code and observe the output. Ask yourself how the output relates to the given data. Use the step functionality in the MARIE simulator to understand step by step what the code does. Which parts are subroutines? Which parts are loops? What do the SkipCond instructions do? How are the different data values used? There may be codes that are "distractors" that do not contribute to the actual logic of the program.

### Task 2.3: Add comments (4 marks)

Comment the code based on your understanding of what it does. Comments should describe the overall function of the program, as well as the functions of different parts. E.g., if you identify a subroutine in the code, add a comment at the start of the subroutine that describes what it does, and whether it takes any arguments.

**For this part (Part 2), you need to submit a ZIP file containing a single .mas file containing your final code. If you attempted all three tasks, your .mas file should *not* contain three copies of the code - only submit the Task 2.3 code in that case (or only Task 2.2 if you did not attempt Task 2.3).**

**The Moodle submission activity for Part 2 contains a “quick check” function that you can use to check if your submission has the right structure.**

## **Part 3: MARIE Programming (35 marks total)**

In this task you will develop a MARIE application that paints characters on the screen. We will break it down into steps for you.

Each task requires you to write **code** and **documentation**. On Moodle, you will find a **template** for the code. **Your submission MUST BE based on this template**, i.e., you must add implementations of your own subroutines into the template. The template already contains the **main program** that calls the subroutines.

**Your code must contain readable comments and meaningful labels** for your tutor / marker to understand the logic flow of your program (e.g., the **purpose** of a subroutine, `Jump / SkipCond` instruction etc.).

**In-class interview (Part 4):** You will be required to join an interview to demonstrate your code to your tutor during your applied session in Week 8 (after the submission deadline). **Failure to demonstrate will lead to zero marks being awarded for the entire Part 3, regardless of your submission in Moodle.** In addition, during the interview (Part 4), you will also need to answer further questions about your submitted code (see below for details).

**Code similarity:** We use tools such as MOSS and Turnitin to check for collaboration and copying between students. If you copy parts of your code from other students, or you let them copy parts of your code, this will result in a report to the Academic Integrity team. As a result, **you may receive a penalty such as 0 marks for the entire assignment, 0 marks for the whole unit, or in severe cases (such as contract cheating), suspension or expulsion from Monash University.**

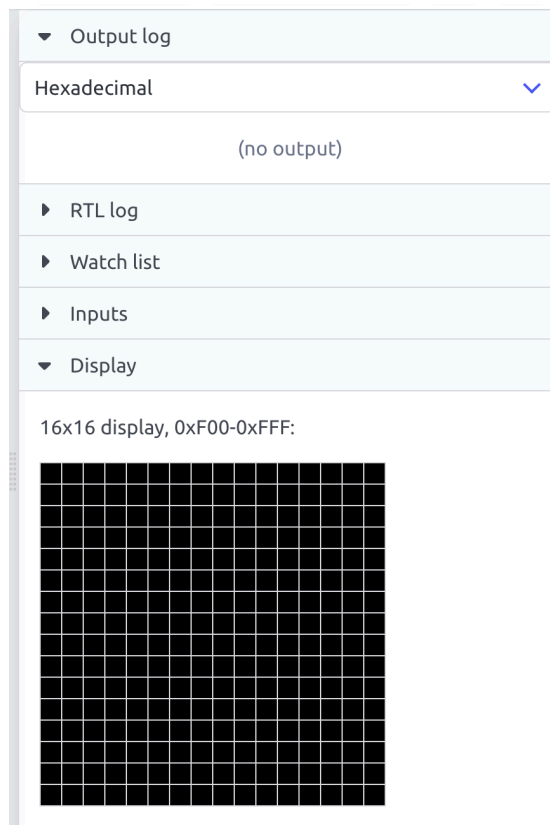
### Introduction: Bit-mapped displays

So far, the only output capability we have seen in the MARIE system is using the Output instruction, which will print a single 16-bit value. Many computers of course are capable of displaying arbitrary graphics, often in high resolution and great colour depth. In the lectures on input/output systems, we have seen that one way to implement this is to **map** a certain location of the memory to an output device, i.e., writing to that memory location (using, e.g., a Store instruction) causes the output to happen.

In the simplest form of graphics hardware, we can dedicate part of the RAM to be **graphics memory**. Each memory cell corresponds to a **pixel** on screen, and the value in the memory

cell encodes the **colour** of the pixel. That way, we can create arbitrary graphics by simply writing values into the memory.

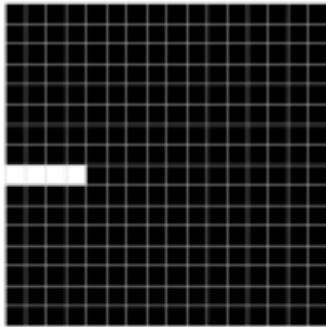
The MARIE simulator has a feature called **Display**, which you access from the list of tabs that also shows the output log, RTL log etc:



The display shows the memory from address F00 to address FFF as a 16x16 pixel screen. The value in the memory locations represents the colour of the pixels. We will only use the colours black, represented as 0, and white, represented as FFFF. When you start the MARIE simulator and assemble your code, the memory starting from location F00 is (usually) filled with zeroes, which means that the display is black. Let's now change the contents of the memory using some Store instructions:

```
Load White
Store 0F80
Store 0F81
Store 0F82
Store 0F83
Halt
White, HEX FFFF
```

After running this program, the display will look like this:



You can see that the first four pixels in the 9th row have now turned white.

### Task 3.1: Clearing the display (4 marks)

Write a subroutine `SubClearDisplay` that uses a loop to turn all pixels in the graphics memory white. Remember that the graphics memory ranges from address `0F00` to address `0FFF`, and that white pixels are represented by the value `FFFF`. Document your subroutine with comments.

### Task 3.2: Inputting a string (8 marks)

Implement a subroutine `SubInputString` that reads an input string provided by the user and stores it in the memory. A string of characters is represented in memory as consecutive locations, each containing an **ASCII code** of a character. In the template, at the end, we have

```
StringAddr, ADR InputString
InputString, DEC 0
```

Store the input string into the memory starting from `InputString`. The format of the input string has to be a sequence of alphabets (i.e., uppercase or lowercase letter(s)) and terminated by a non-alphabetic character. For example, **secUrity%**, **cRypTogrAphy&**, and **fIT3**. The non-alphabetic character is a delimiter that allows your program to determine the end of your input string. In the subroutine, before storing the input string, convert the string to uppercase letters first. For example, if the input is **secUrity%**, **SECURITY%** will be stored in the memory.

Note that if your input is, for example, **tWo^WoRdS**, only **TWO^** (the upper case of **tWo**) is stored into the memory, because the first non-alphabetic character, **^**, is encountered before the rest of the alphabets, **WoRdS**.

Again, your subroutine needs to contain sufficient comments to enable someone else to understand the purpose of each line of your code.

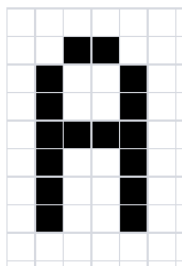
### Task 3.3: Painting a character (6 marks)

The template for this task contains data for **bitmaps** of the letters 'A'-'Z', stored at the label `Font`. Each letter consists of 4x8 pixels of data. The first 4 words are the first row of pixels, the next 4 words are the second row, and so on. For example, the letter 'A' is represented as

```

FFFF 0000 0000 FFFF
0000 FFFF FFFF 0000
0000 FFFF FFFF 0000
0000 0000 0000 0000
0000 FFFF FFFF 0000
0000 FFFF FFFF 0000
0000 FFFF FFFF 0000
FFFF FFFF FFFF FFFF
  
```

You can see the pattern here, the zeros “paint” the shape of the character ‘A’. We have highlighted the zeros in bold and red to make the pattern easier to see, but on the real display, the zeros would be black against the white background (FFFF).



Your task is to write a subroutine called SubPaintChar that paints a character into the graphics memory. The start of the subroutine needs to look like this:

```

PaintCharCharacter, HEX 0
PaintCharDisplay, HEX 0
SubPaintChar, HEX 0
  
```

In the PaintCharCharacter argument, we pass the address of the first pixel data in the font for the character we want to paint. In the PaintCharDisplay argument, we pass the address of the top-left corner where we want to start painting in the graphics memory. For example, to paint the character ‘C’, starting from the second pixel in the second row, we could use the following code:

```

Load FontAddr
Add SIXTYFOUR
Store PaintCharCharacter
Load Display22
Store PaintCharDisplay
JnS SubPaintChar
Halt
Display22, HEX 0F11
SIXTYFOUR, DEC 64
  
```



Note that the address 0F11 (label Display22) lies exactly 17 words after the start of the graphics memory. This means we're skipping the first row (16 words) and the first pixel in the second row (1 word).

Here we use FontAddr, which refers to the first character ('A'), and then add decimal 64 to it, to get the address of the character 'C' (each character uses  $4 \times 8 = 32$  words of memory, so 'C' starts at FontAddr + 64). For the other characters, we would have to add a corresponding offset into the font memory.

In order to paint a character in your subroutine, you can follow this "recipe":

- Your subroutine should contain two nested loops.
- Each character contains 32 pixels, so you need to loop through those 32 pixels, load each one from the font definition and store it into the graphics memory. This is the outer loop of your subroutine.
- After each set of 4 pixels (i.e., 4 columns in a row), you need to start in the next row of the graphics display. This means that if you were currently writing into graphics memory at address X, you now need to continue writing at address X plus the width of the display minus the width of a character. This is the inner loop of your subroutine.
- Once you have "copied" all 32 pixels from the font definition into the graphics memory, you can exit the subroutine.

Your subroutine needs to contain sufficient comments to enable someone else (like the person marking your assignment) to understand the purpose of each line of your code.

#### Task 3.4: Writing a string (10 marks)

Your final task is to implement a subroutine SubPaintString that reads an input string, stores the string to the memory, clears the screen and then writes the string (excluding the delimiter) to the display, one character, at a time. Each character of the string is painted using the subroutines developed in the previous tasks. After each character has been painted, you need to clear the screen again and then paint the next character.

The starting address for the string of characters is passed to the subroutine:

Load StringAddr

Store StringAddrForPainting

JnS SubPaintString

The string can be painted anywhere on the display you like.

You will notice that it would be nice for each character to remain visible for a second or so before the next character is painted. Think of a way you can achieve this. Document how you achieved this in the code comments.

A video with sample input/output cases for this part is provided for your reference on Moodle.

**For this part (Part 3), you need to submit a ZIP file containing the .mas file with the code for all subroutines, based on the template. Do not submit multiple .mas files!**

Task 3.5: Be creative (7 marks, only available if Tasks 3.1 - 3.4 are complete and correct)

For the final task, think about creative ways to use the code you developed for Tasks 1 - 4. For example, you could make the text scroll or jump across the display, have some fun with colours, or any other idea you would like to implement. This task will be marked based on **creativity** (such as colours, text scroll and gamification etc.) and **technical difficulty** (such as usage of additional subroutines, boundary check, parameter passing etc.).

Only work on this task when you have completed Tasks 3.1 - 3.4. **You will not be marked for Task 3.5 if the code you submit for Tasks 3.1 - 3.4 is incomplete or incorrect. Submit a separate (self-contained) .mas file for this part, inside the same ZIP file that contains your submission for Tasks 3.1 - 3.4.**

**You may implement additional supporting subroutine(s) and create new label(s) of memory location(s) for Tasks 3.1 - 3.5.**

## **Part 4: In-class interview (10 marks total)**

You need to demonstrate the code you submitted for Part 3 Task 3.1 - 3.4 to your tutor in an in-class in-person interview (**to be conducted during your official allocated Applied session in Week 8**) after the submission deadline. Failure to explain how your code works will result in 0 points for the individual tasks that you cannot demonstrate.

In addition, you will be asked questions about the following topics:

- Modify your MARIE code in certain ways to make it behave differently.
- Explain how the MARIE concepts work that you were required to use for the individual tasks.
- Demonstrate how to access a list of processes and applications on your computer.
- Answer questions about processes, memory and I/O as covered in the lectures.

These additional questions add up to 10 points for this part.

**Failure to attend the interview will result in 0 points for the entire Part 3 and Part 4, regardless of your submission in Moodle.**

**Submission Checklist: Please check before you submit.**

**Altogether, you should submit the following files:**

**Part 1:**

One pdf file for Week 4 to Week 6 Reflections

**Failure to submit this file will result in a maximum of 50% of marks in this assignment.**

**Part 2:**

One .zip file containing one .mas file with your final code for Tasks 2.1-2.3. Do **not** submit one .mas file *per each subtask*. Do **not** submit any pdf file in this part. Use the “quick check” function on Moodle to check that your submission matches the expected format.

**Part 3:**

One .zip file containing one .mas file with your final code *based on the template*, containing the code for all subroutines. Do **not** submit one .mas file *per each subtask*. Do **not** submit any pdf file in this part. Optionally, one additional .mas file inside the same .zip file for Task 3.5.

**Failure to submit the correct type of files (e.g., submit a word file instead of a pdf file in Part 1, or a pdf file in Part 2 or Part 3), or missing any file in your submission (e.g., your weekly reflection) will result in significant marks deduction.**

**Please check before you submit.  
You CANNOT revert any submission.**

**Draft submissions are not assessed.**